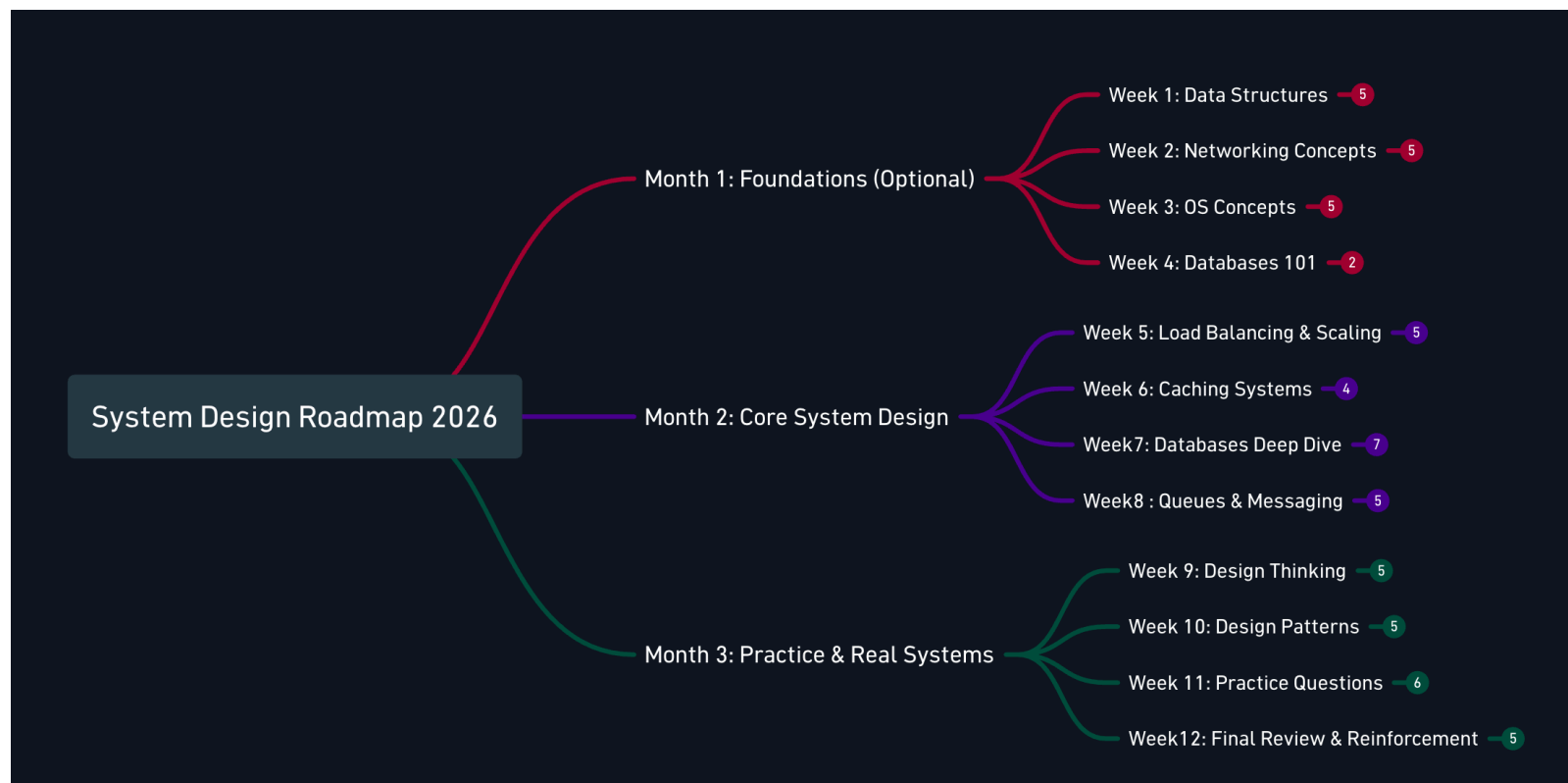
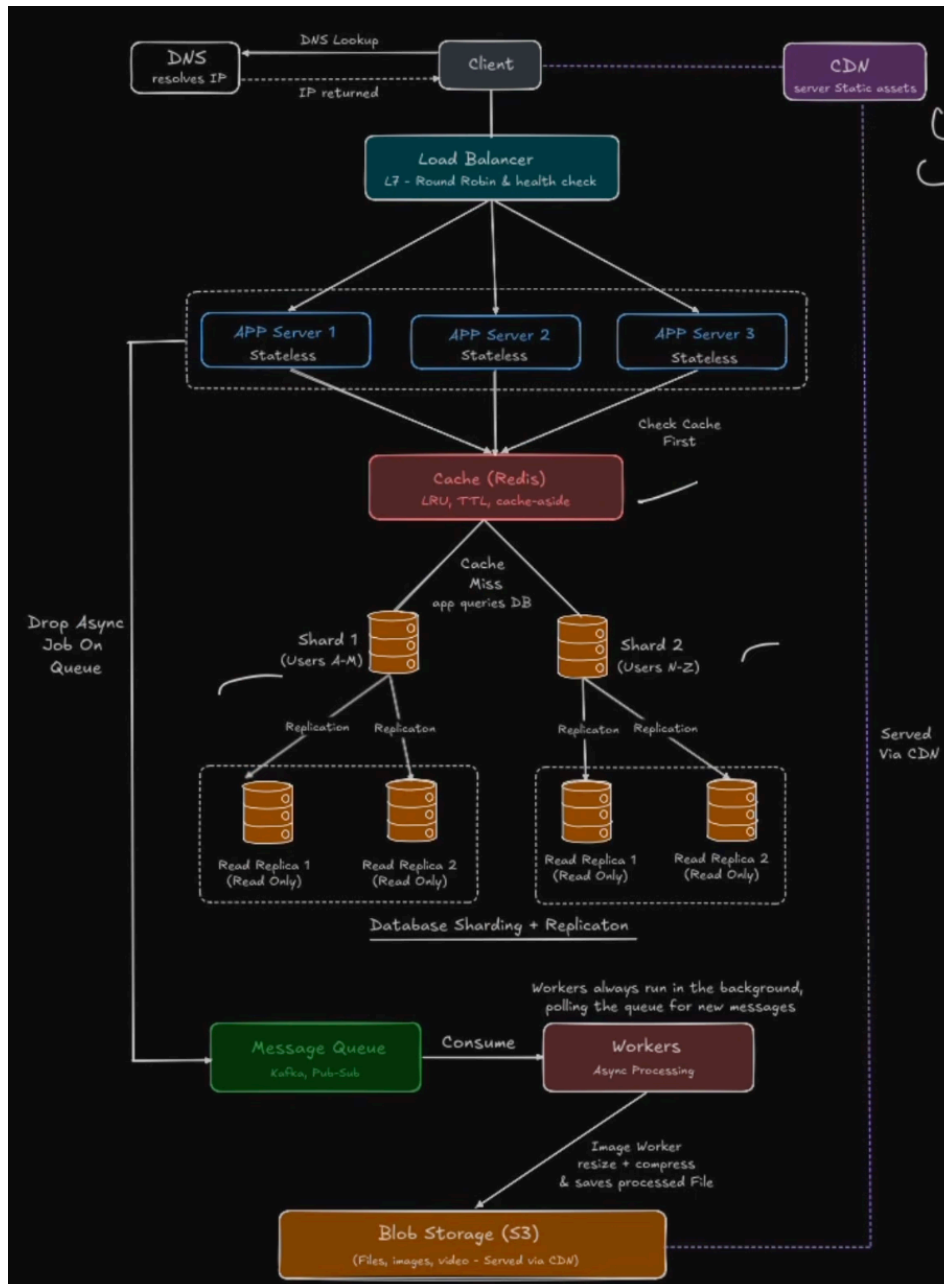


System Design - Terminologies

Roadmap:

1. *Foundation* - what, computation, components, functional and non functional aspects
2. *Communication* - DNS, API's, http, RESTful suite
3. *Data Layer* - Sql, no Sql, Cache
4. *Scaling and distribution* - CAP, load balancer, replication, partitioning
5. *Reliability and Operations* - Message queues, fault tolerance, Monitoring
6. *Case Studies* - in different notes.





Analogy to Cashier-Customer:

How can we improve this ?

- Improve skills of cashier.
- Suppose cashier has 3 tasks (money, receipt, checkout) - we must delegate these tasks to different people.
- Increase number of cashiers and their delegations as well - but in this database must be central
- balancing load on all the cash counters - equally spread in ideal case.

Mapping to system design:

- Improve code quality - TC and SC, maintainability, extensibility - LLD and DSA.
- Vertical Scaling - improving the single server that we have.
- Horizontal Scaling - adding more servers.
- Load balancers - delegating the tasks to each server properly.

Types of Architecture:

1. Monolith -
2. Distributed / micro services -

Synchronous and Asynchronous calls:

if we wait for some service to complete and respond back - Synchronous call.

if we do not wait for some service to complete and respond back - Asynchronous call.

Message queues are used for handling asynchronous tasks.

Types of Applications: (Both have latency issue - what we have to fix)

1. Data Intensive Apps - less maths, more fetch calls from databases - optimisations in Disk I/O, Network calls, Memory.
2. Compute Intensive Apps - more maths, less data from databases - GPU bounds - optimisations in GPU/CPU.

Data-Intensive Application Characteristics



Continuous Data Flow

Large volumes of data are read and written continuously.



Network Calls

Multiple network calls are involved in serving requests.



Speed Focus

The focus is on delivering data from the database to the client as fast as possible.

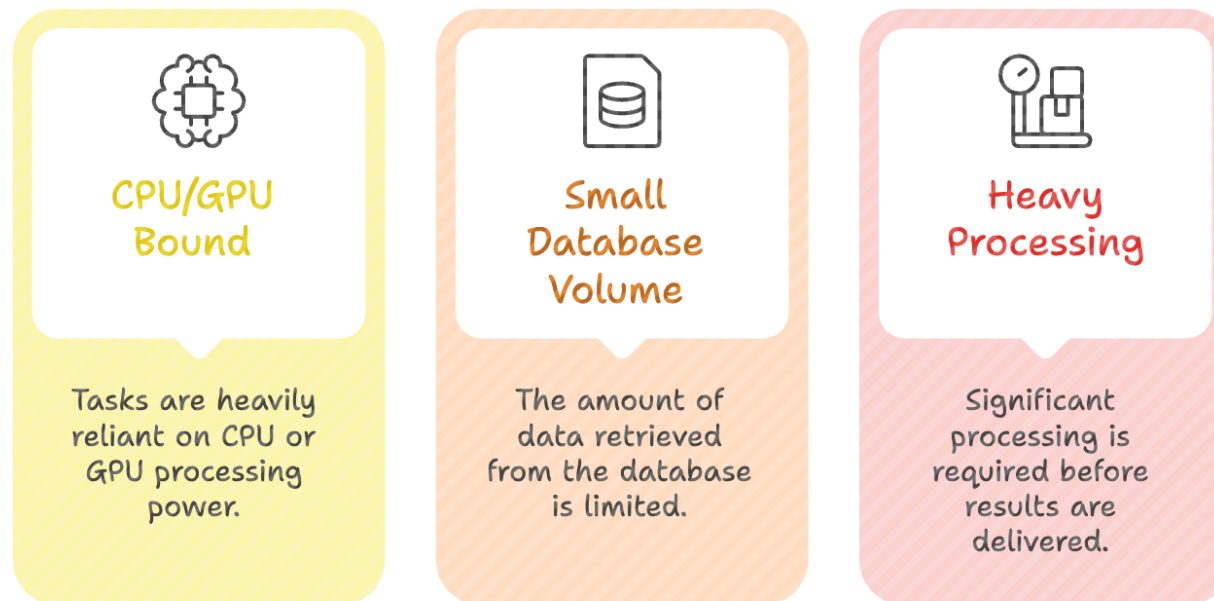


Bottleneck Identification

Upgrading client-side CPU won't help — the bottleneck is in data access and transfer.

Solved by - caching, Sharding, replication, consistency, DB optimisations.

Compute-Intensive Application Characteristics



eg - image processing, ML Model training, Cryptography, video rendering etc.

Solved by - Parallelism, GPU acceleration, algorithm optimisation

Types of Requirements:

1. Functional - user functions, features of app.
2. Non-Functional - uptime defined in SLA/SLO, scalability, fault tolerance, estimations of numbers, auto scaling, security, latency, authorisation, maintainability, reliability, observability.

How do you know that your design is good enough?

1. Have you divided system into components?

2. Every component has clear set of responsibilities?
3. Figure out details like database, caching, fault tolerance, async/sync communication.
4. is each component in isolation scalable, fault tolerant and reliable? No SINGLE POINT OF FAILURE (SPOF).

Domain Name System:

We cannot store all the websites domain names map to IP addresses in our systems or browsers as the data is too large so we have DNS for resolving this issue.

Issues:

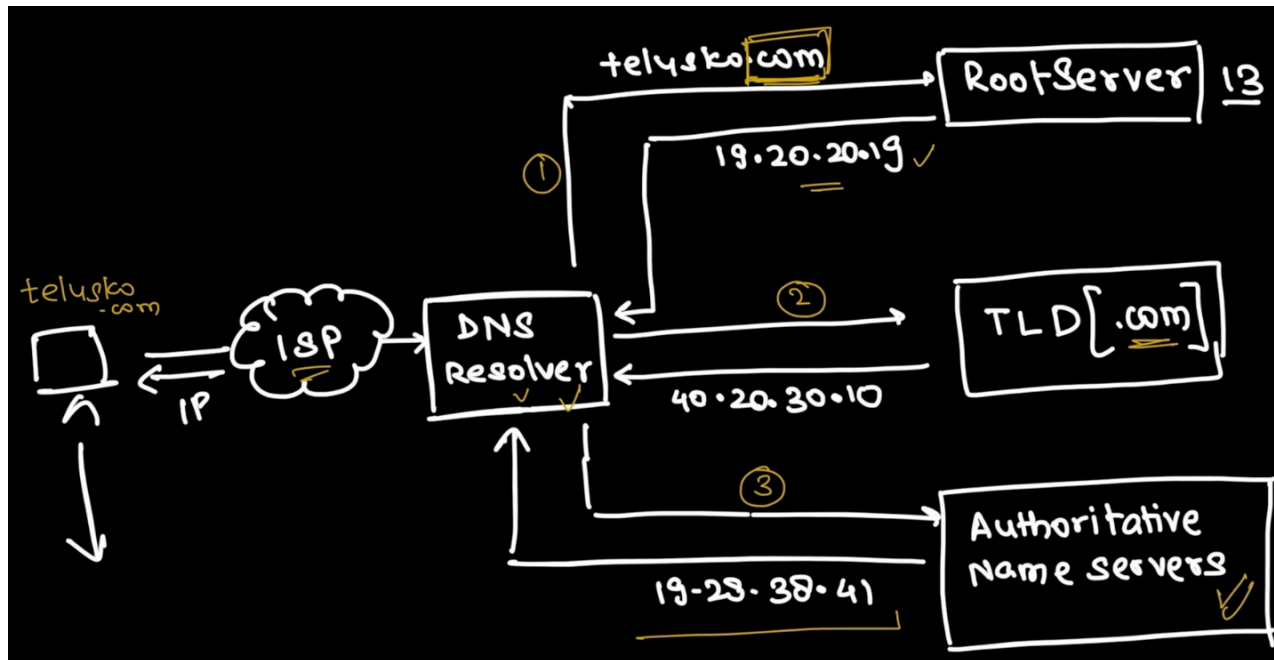
1. Storage
2. Modification of IP address maintenance.

DNS Resolution Process



Steps are as follows:

1. Your machine connects to the internet through an Internet Service Provider (ISP), which has its own DNS resolver.
2. There are 13 root servers worldwide, named a.root-server.net through m.root-server.net
3. The root server does not know the final IP address - it directs the resolver to the appropriate TLD server.
4. TLD are top level domains example - .com, .net, .in, .co.in etc.
5. The TLD server responds with the IP address of the Authoritative Name Server that manages the specific domain.
6. ANS holds the actual DNS records for the domain.
7. Returns the final IP address of the requested domain to the DNS Resolver.
8. The DNS Resolver returns the IP address to the browser.



DNS Resolution Process



User Input

The user types the website address into their browser (telusko.com)

The user's ISP's DNS resolver receives the request

DNS Resolver



Root Server

The DNS resolver queries the root server for the .com TLD server's IP address (a-m.root-server.net)

The root server returns the IP address of the .com TLD server

TLD Server



Authoritative Name Server

The TLD server returns the IP address of the authoritative name server (telusko.com)

The browser connects to the IP address of telusko.com, and the webpage loads

Browser Connection



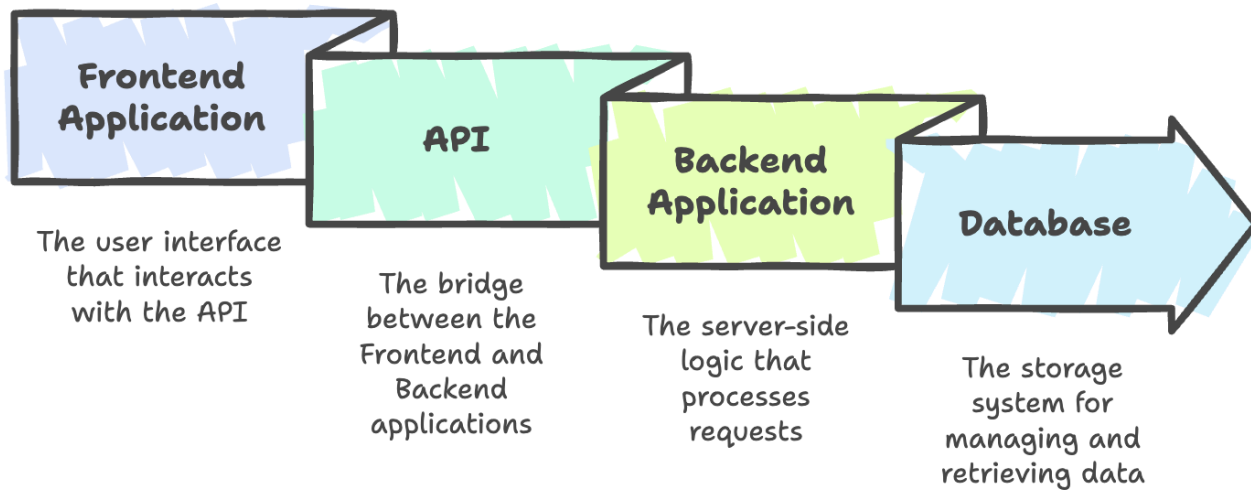
Caching in the whole process of DNS Resolution. There are zones for subdomain resolution in ANS part.

Cache Levels (checked in order)

Level	Location	Description
1	Browser Cache	Browser stores recently resolved domains locally
2	OS Cache	Operating system maintains its own DNS cache
3	DNS Resolver Cache	The ISP's resolver caches results for all users

API's - Application Programming Interfaces:

It is a set of endpoints exposed by an application that allows other applications (or frontends) to communicate with it, access its data, and perform operations without needing direct access to the underlying database.



If we give access to applications or frontends without APIs then it can be dangerous for our data in DB. So APIs restrict or control this access.

Types of APIs:

1. REST APIs - Representational State Transfer - data format JSON - uses HTTP methods.
2. SOAP APIs - Simple Object Access Protocols - data format XML - used for cross platform communication.
3. Graph QL - Graph Query Language - frontend can request multiple table's data in single query - data format JSON - solves issue of multiple requests (under-fetching) and over-fetching.
4. Web sockets - Real time - two way communication between client and server.
5. gRPC - google remote procedure call - data format protocol buffers - small in size so faster - micro service to micro service communication.

API Type	Data Format	Best For	Latency	Complexity
REST	JSON	General-purpose web APIs	Low	Simple
SOAP	XML	Legacy/enterprise systems	Moderate	High
GraphQL	JSON	Flexible frontend queries	Low	Moderate
gRPC	Protobuf	Microservice internal communication	Very Low	Moderate
WebSocket	Any	Real-time bidirectional communication	Real-time	Moderate

RESTful APIs:

These are standard format for transferring data from clients to servers, designing these depends upon resources.

endpoint = http method + path.

endpoints is location from which we can access on API.

Method - Defines the operation (GET, POST, PUT, PATCH, DELETE)

Path - The URL through which data is accessed or submitted

Requests in REST APIs:

Methods:

Method	Path	Purpose
GET	/users	Retrieve all users
GET	/users/{id}	Retrieve a specific user by ID
POST	/users	Create a new user (data sent in body)
PUT	/users/{id}	Replace the entire user record
PATCH	/users/{id}	Partially update specific fields
DELETE	/users/{id}	Delete a user

A REST API request consists of - method + path url + headers + body (optional)

headers - contains meta data, tokens, permissions for the request.

path url - tells api which resource to access.

Some more concepts:

1. Nesting - When entities have relationships. eg - /blogs/{id}/comments
2. Filtering - uses query parameters to narrow down results based on conditions. eg - /comments?post_id=123

There are three ways to send data in a REST API request:

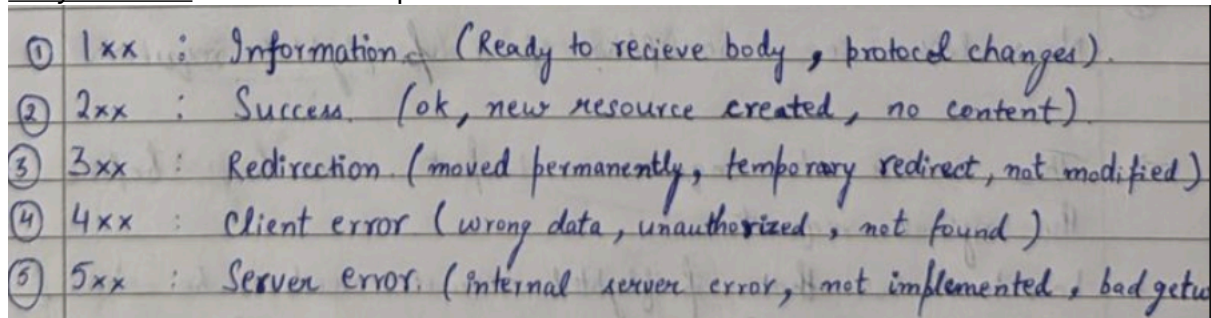
Method	Use Case	Visibility	Example
Path Parameter	Pass IDs, slugs, or unique identifiers	Exposed in URL	/users/42
Query Parameter	Filtering, sorting, pagination	Exposed in URL	/users?age=20&sort=name
Request Body	Sensitive or complex data (credentials, payloads)	Hidden from URL	{ "username": "muskan", "password": "***" }

Responses in REST APIs:

corresponds to what server sends for each client request.
consists of status code + body returned.

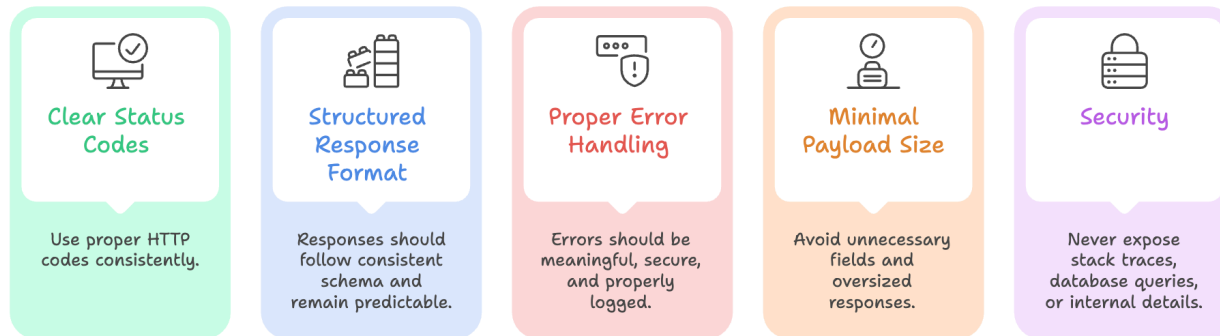
status codes - for real better understanding and documentation clarity.

body returned - the actual response from server.



①	1xx	: Information (Ready to receive body, protocol changes)
②	2xx	: Success (ok, new resource created, no content)
③	3xx	: Redirection (moved permanently, temporary redirect, not modified)
④	4xx	: Client error (wrong data, unauthorized, not found)
⑤	5xx	: Server error (internal server error, not implemented, bad getuo)

Characteristics of Good API Responses



Databases:

do it from DBMS notes for detailed explanations.

Database Scaling.

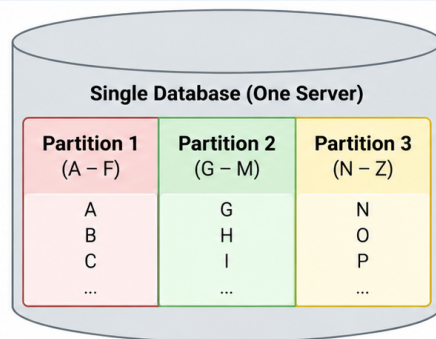
Scaling reads - Partitioning
Scaling writes - Sharing

Sharding: method of distributing data across multiple machines

Partitioning: splitting a subset of data within the same instance

Partitioning

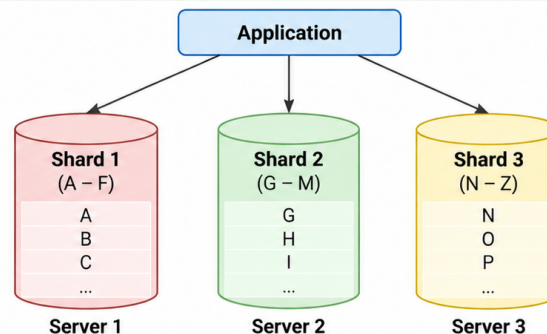
Partitioning splits a single large database (or table) into smaller parts (partitions), but all partitions reside on the **same server/instance**.



- Data is split into partitions (e.g., by range, hash, or list).
- All partitions are stored on the same database server.
- Helps with manageability and query performance.
- Does not scale to multiple machines (still one server).

Sharding

Sharding splits a large database into smaller, independent databases (shards) that are distributed across **multiple servers/machines**.



- Data is split into independent shards (e.g., by range, hash, or tenant).
- Each shard is stored on a different server/machine.
- Helps with horizontal scalability (can handle more data and load).
- Each shard works independently.

Caching:

What is cache?

Any memory/data that lets us avoid computation is known as cache.
Without caching - load on server increases, more latency to users.
Generally stored as Key-Value pairs.

These can be at certain places:

frontend - client side cache
backend - server side cache
database - WAL or DB cache

Some other terms:

cache hit - when the requested data is already present in the cache.

cache miss - when the requested data is not available in the cache.

TTL - Time to live (the time for which an entry will be auto deleted from cache)

Strategies of keeping updated cache:

1. Read Through Cache (RTC) - read from cache, write to DB and in case of cache miss out to cache.
2. Write Through Cache (WTC) - read from cache, write in cache and cache writes to DB. eg - stock prices
3. Write Around Cache (WAC) - read from cache, write always to DB. eg - Instagram posts.
4. Write Back Cache (WBC) - write to cache but write to DB is asynchronous so performance > consistency.

Strategy Comparison				
Strategy	 Read-Through	 Write-Through	 Write-Around	 Write-Back
Write Target	Database	Cache → Database (sync)	Database	Cache → Database (async)
Read Target	Cache	Cache	Cache	Cache
Consistency	Moderate	Strong	Low (risk of stale data)	Eventual
Best For	Read-heavy workloads	Data requiring latest values	Infrequently read after write	High write traffic, speed-critical

Cache Eviction Policies:

1. Least Recently Used
2. Most Recently Used
3. Least Frequently Used
4. First in First out
5. Last in First out

Cache Eviction Policies					
Policy	 LRU	 MRU	 LFU	 FIFO	 LIFO
Evicts	Least recently accessed	Most recently accessed	Least frequently accessed	Oldest inserted entry	Newest inserted entry
Based On	Last access time	Last access time	Total access count	Insertion order	Insertion order
Best For	General-purpose caching	Data unlikely to be reused	Workloads with clear popularity	Simple, predictable eviction	Specific use cases

Load Balancer:

It is a component that delegates the incoming client's requests to the most appropriate server base on algorithms to improve fault tolerance, performance, scalability and availability.

Types:

based on layer ->

1. layer 4 - transport layer, fast, content packet is not impt,
2. layer 7 - application layer, slow, content packet is impt,

based on routing ->

1. static - routes based on pre defined algorithms

2. dynamic - routes based on server's health checks and performance, difficult to configure.

Tasks:

1. Delegation - routing to server in most efficient way.
2. Health Checks - should know which server has ability at present.

Algorithms for delegation:

Generally multiple approaches are combined together for best outcomes.

1. Round Robin - equal distribution, one by one request, issue when servers are not equally capable.
2. Weighted Round Robin - weight assigned on each server based on configuration of the server.
3. Geo-Based - req routed based on geographic location of server, can be misled with VPNs, overhead of database replication to each location.
4. Least Connection Algo - number of connections, but not a good matrix when servers have different capacity.
5. Least Time - routes to lowest average response time, overhead of maintenance of reqs/sec. ideal for stocks system
6. IP Hash - client's IP is hashed to check for which server to route, same client same server -> can be cached, challenging to add new server -> rehashing needed.

Algorithm	Basis of Decision	State Maintained	Best For
 Round Robin	Sequential order	Counter only	Equal-capacity servers
 Weighted Round Robin	Server capacity	Weights	Heterogeneous server configurations
 Geo-Based	Geographic proximity	Location data	Globally distributed users
 Least Connection	Active connections	Connection count	Variable request durations
 Least Time	Response time	Avg response time	Latency-sensitive applications
 IP Hash	Client IP hash	Hash mappings	Session persistence

Health checks:

Health checks are mechanisms used by a Load Balancer to continuously monitor the availability, responsiveness, and overall health of backend servers.

By doing this we know about availability, reliability of response, user experience, metrics for future scalability.

Parameters:

1. Interval - Frequency at which health checks are performed.
2. Timeout - Maximum time the load balancer waits for a response.
3. Threshold - Number of consecutive successes or failures required before changing server status.

Types:

1. Passive Health Checks - No extra requests, checks from the real users responses.
2. Active Health Checks - dedicated requests are sent for checks, accurate but adds a small amount of traffic as well.

Database Replication:

Replication is the process of copying and maintaining data across multiple databases or nodes. Instead of storing data in a single location, identical copies are maintained on multiple servers.

Done for high availability, fault tolerance (avoids SPOF - single point of failure), high read throughputs.

Types of Replication:

1. Single Leader Replication - one leader, others are followers, only leader accepts write requests, async replication (can be done sync as well), this may give faster responses but has temporary consistency issues for the time DB is not updated. In case of leader failure -> the replica having latest updated is elected as leader.
2. Multi Leader Replication - multiple data centres each having their own leader, leaders are given priority based on which data consistency is maintained.
3. Leaderless Replication - every node can receive read and write requests,

Partitioning:

Partitioning is the process of dividing a large dataset into smaller, manageable chunks called partitions, which are distributed across multiple nodes. Improves scalability (distributed storage) and performance (by parallelism in query execution).

Problem - when one partition receives more requests it becomes hotspot.

Partitioning Strategies:

1. Partition by Key - based on range of key values. hotspots when traffic is unevenly distributed -
2. Partition by Hash of Key - data stored in one partition has same hashed value.
3. Partition by local secondary index - each partition has it's own secondary index, so when data needs to be fetched we must query ALL partitions.
4. Partition by global secondary index - 1 centralised index for all partitions, here only required partitions are queried, NOT ALL.

Strategy Comparison

Strategy	Distribution	Hotspot Risk	Read Efficiency	Write Efficiency
 Key Range	Based on key ranges	High (skewed access)	Good for range queries	Simple
 Hash of Key	Based on hash values	Low (uniform)	Good for point queries	Simple
 Local Secondary Index	Per-partition indexes	Moderate	Scatter-gather (all partitions queried)	Simple
 Global Secondary Index	Centralized index	Low	High (targeted queries)	Complex (index update overhead)

CAP Theorem:

A distributed system cannot simultaneously guarantee all three properties: Consistency (C), Availability (A), and Partition Tolerance (P). It can guarantee only two of these three properties at any given time.

Property	Full Form	Meaning
C	Consistency	Every user always receives the latest and most up-to-date data from the system.
A	Availability	Every request receives a response, even if some nodes are experiencing failures.
P	Partition Tolerance	The system continues to operate even when communication failures or network partitions occur between nodes.

CAP Theorem Combinations

Combination	Guarantees	Sacrifices	Example
 CA	Consistency + Availability	Partition Tolerance	Small single-database apps
 CP	Consistency + Partition Tolerance	Availability	Banking, financial systems
 AP	Availability + Partition Tolerance	Consistency	Instagram, social media

Message Queue (MQ):

A Message Queue is a communication mechanism that enables asynchronous communication between applications or services. It acts as an intermediary (buffer) that temporarily stores messages sent by one service until another service is ready to process them. Instead of requiring two services to communicate directly, the producer sends messages to the queue, and the consumer retrieves and processes them independently.

Message queues improve scalability, reliability, fault tolerance, sudden traffics and service decoupling in distributed systems. There are 2 types of communications:

1. Synchronous - we have to wait for response - finance apps, security, DB transactions.
2. Asynchronous - no waiting - email service, event driven calls, analytics pipelines.

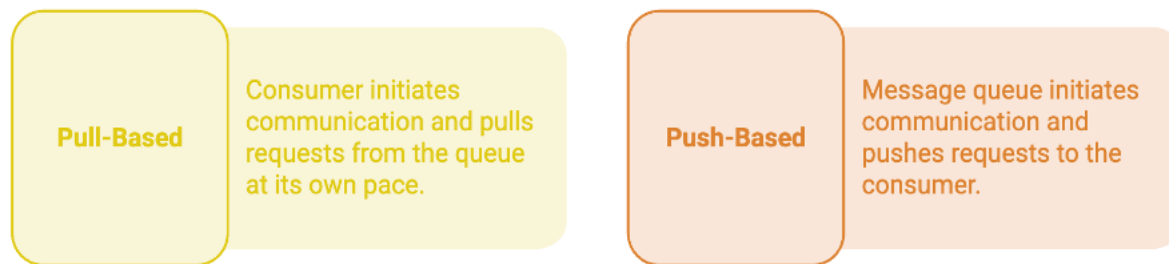
Participants:

1. Producer / Publisher
2. Consumer / Subscriber
3. MQ / Broker

Processing Approaches (queue -> FIFO):

1. Strict Order Queue - waits for failed req to resolve then moves to next one.
2. Unordered Queue - skips the failed req and moves to next one.
3. Priority Queue - high priority are processed before low priority.

Message Queue Models



Pub-Sub Model:

This consists of multiple producers to publish and multiple consumers to consume independently.

Steps:

1. Publishers send messages to the queue.
2. The queue stores the messages.
3. Subscribers consume the messages independently.

4. The queue monitors subscriber health.
5. Failed deliveries are retried automatically.

Key factors for pub-sub:

Factor	Description
Message Order	Can be maintained using FIFO or Priority Queues depending on the implementation.
Message Consumption	Subscribers consume messages independently, and processing order may vary.
Poison Message	A message that repeatedly fails processing due to invalid data, bad requests, or consumer issues.
Duplicacy	Successfully processed messages must be removed from the queue to prevent duplicate processing.

Dead Letter Queue (DLQ):

Secondary queue to store messages that have been retried after multiple fail attempts.

This prevents poison messages from blocking the main queue, easy to debug and troubleshoot, manual testing can be done.

Faults and Errors:

A fault or error occurs when a system fails to perform as expected or deviates from its intended behaviour. Faults can affect a system's reliability, availability, performance, and data integrity.

Categories of Faults:

1. Hardware - physical infrastructure of a system - random, unpredictable, hard to prevent - as hardware wears out at some time redundancy and data replication are major solutions. eg - disk full / crash, cable, server damage.
2. Software - issues in application code, system configuration, deployment, or software logic - edge cases, deployments, merge conflicts - preventable by unit testing, code reviews, exception handling, CI/CD.
3. Human Errors - incorrect actions or decisions made by users, developers, administrators, or operators - most unpredictable sources of system failures - prevented by RBAC, documentations, automation of repetitive tasks.

Category	Nature	Predictability	Mitigation
 Hardware	Physical infrastructure	Random	Redundancy, replication, monitoring
 Software	Application level	Identifiable	Code reviews, testing, proper handling
 Human	Human actions	Unpredictable	Automation, access controls, training

Monitoring and Observability:

These are practices used to continuously track the health, performance, and behaviour of a system. Helps devs to quickly detect and resolve issues but is only useful when proper alerting is done.

Monitoring	Observability
Tracks predefined metrics, logs, and alerts	Helps understand the internal state of the system using metrics, logs, and traces.
Answers "Is something wrong?"	Answers "Why is something wrong?"
Focuses on known errors/problems	Focuses on unknown errors/problems
Common metrics: CPU utilisation, API response time, memory consumed, error rates, server capacity.	Common metrics: complete cycle of request to response through different micro services.

API Monitoring:

focuses on performance, availability and reliability of application endpoints (apis).

Components to be monitored ->

1. Throughput - the number of requests per unit time. determines load and scaling decisions.
2. Error Monitoring - tracks response codes for unsuccessful requests. determines failures in app & deployments
3. Health Checks - continuously verify if a service or component is healthy or not. determines if load balancer should route request to it or not, reliability.
4. Latency - the time required for a system to respond to a request. determines user experience and performance.

More on Latency Estimations -

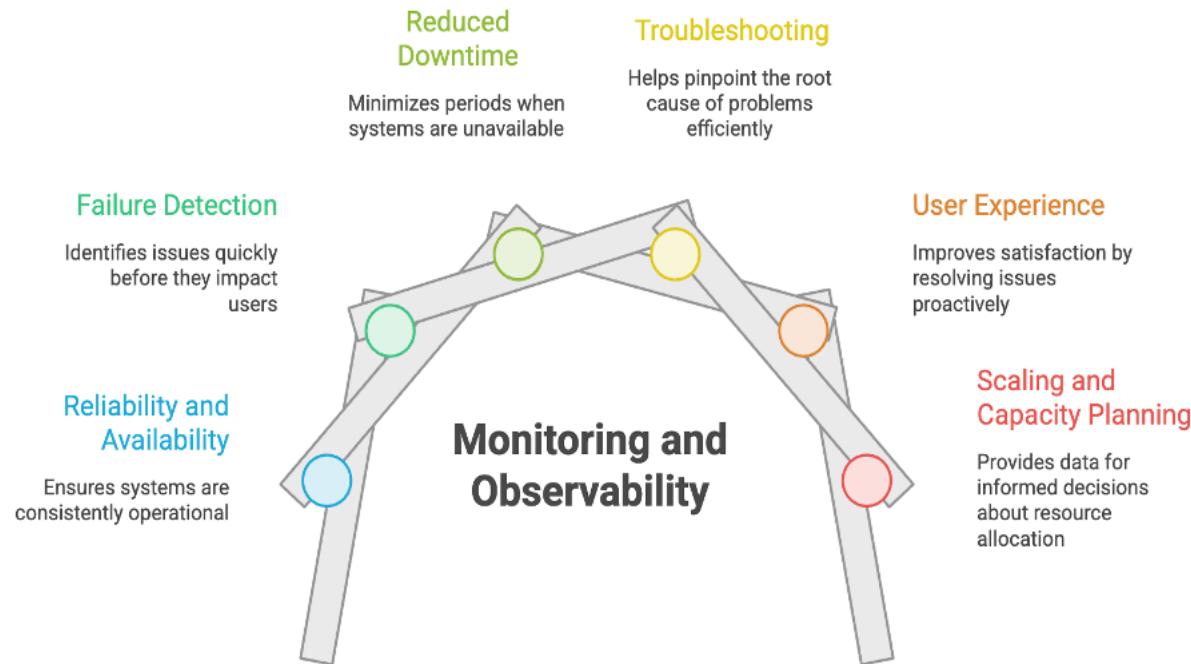
Averages are not true measure therefore we measure in percentiles.

A P90 latency of 200 ms means:

- 90% of requests complete within 200 ms (can be less as well).
- 10% take longer than 200 ms.

System Monitoring:

1. CPU Usage - determines performance bottlenecks, notify when set threshold is met.
2. Memory Usage - tracks RAM consumption, determines out of memory, resource allocation.
3. Disk I/O - read and write operations monitoring, determines storage and slow DB operations.
4. Network I/O - incoming and outgoing traffic, determines cyber attacks, communication b/w services.



Other topics to be covered:

Rate Limiting - Token Bucket, Leaky Bucket, Fixed Window, Sliding Window, Distributed rate limiting
Consistent Hashing
Consistency Models - strong eventual RAW monotonic casual
Proxies & API Gateways
CDN
monolith architecture
Idempotent meaning and requests.
Optimistic vs. pessimistic locking, distributed locks
Server Sent Events
Kafka
Edge Servers
Connection Pooling
Reliability Patterns - jitter timeout circuit break bulkhead
Database - b/b+ trees

Cache - stampede, avalanche penetration hotkey invalidation
Stateful and opposite
Disaster recovery
2PC in transactions
CQRS
Event Sourcing
CDC - change data capture
Service mesh
Bloom filter
vector clocks
kubernetes intervals
Saga